

Implementación de un agente personal ReAct para la planificación académica y consulta contextual de materiales mediante LangChain

1. Título del proyecto

Implementación de un agente personal ReAct para la planificación de actividades y consulta contextual de materiales académicos mediante LangChain

2. Descripción general

Agente personal basado en el patrón **ReAct** (*Reasoning and Acting*, "Razonamiento y Actuación") que organiza actividades académicas pendientes, calcula cuál atender primero según su fecha límite y prioridad, y consulta los documentos de la carpeta asociada a cada tarea para ayudar a comenzarla (instrucciones, rúbrica, conceptos).

Versión 1 (esta entrega): tareas en un archivo **JSON** (*JavaScript Object Notation*, formato de texto para guardar datos estructurados) y búsqueda de documentos por palabra clave — **sin** *vector store* ni **RAG** (*Retrieval Augmented Generation*, recuperación de información mediante embeddings semánticos). Ver Sección 15 para el porqué de este alcance.

3. Necesidad personal

Durante el desarrollo de cursos, trabajos y proyectos se acumulan actividades con distintas fechas límite y niveles de importancia. Decidir a mano cuál atender primero, y encontrar dentro de qué documento está un requisito puntual, consume tiempo que preferiría dedicar a resolver la tarea en sí.

El agente ayuda a decidir: qué actividad hacer primero, cómo distribuir el tiempo disponible entre las tareas pendientes, y qué dicen los materiales de una tarea sin tener que abrir cada archivo manualmente.

4. Objetivo general

Implementar un agente personal basado en el patrón ReAct que priorice actividades académicas pendientes y consulte los documentos asociados a cada una para apoyar su desarrollo, usando LangChain.

5. Objetivos específicos

1. Registrar, consultar y actualizar actividades académicas en un archivo JSON.
2. Calcular un puntaje de urgencia por tarea, combinando fecha límite y prioridad declarada.
3. Distribuir las tareas pendientes dentro de un bloque de tiempo disponible.
4. Inspeccionar una carpeta autorizada y buscar contenido relevante dentro de sus documentos, por palabra clave.
5. Integrar todo lo anterior en un agente ReAct con `create_agent` de LangChain.

6. Ejemplo de uso

Tengo cuatro horas disponibles. Revisa mis tareas pendientes, dime cuál es más urgente y busca en sus materiales cuáles son los requisitos de entrega.

Ciclo esperado (acción → observación → razonamiento, repetido hasta la respuesta final):

Acción: `consultar_tareas`

Observación: 3 tareas pendientes.

Acción: `calcular_prioridad` (para cada una)

Observación: la tarea "Implementar agente ReAct" tiene el puntaje más alto.

Acción: `generar_plan(minutos_disponibles=240)`

Observación: plan con esa tarea y su duración estimada.

Acción: `inspeccionar_carpeta`

Observación: 2 PDF, 1 DOCX, 1 archivo Python.

Acción: `buscar_en_documentos(consulta="requisitos de entrega")`

Observación: línea encontrada en instrucciones.docx.

Respuesta final: plan priorizado + lo encontrado en los materiales.

7. Modelo de datos — `tareas.json`

```
[
  {
    "id": 1,
    "nombre": "Implementar agente personal ReAct",
    "curso": "Implementación de agentes con IA",
    "fecha_limite": "2026-08-02",
    "duracion_estimada_minutos": 240,
    "prioridad": "alta",
    "estado": "pendiente",
    "ruta_contexto": "C:/MaterialesAcademicos/AgentesIA/TareaReAct",
    "entregable": "Google Doc con objetivo y código"
  }
]
```

prioridad ∈ {alta, media, baja}. estado ∈ {pendiente, iniciada, completada}.

8. Herramientas del agente (código real, no pseudocódigo)

Siete *tools* deterministas — el mismo criterio que `agente_presupuesto_materiales.py` de esta sesión: el **LLM** (*Large Language Model*, modelo de lenguaje grande) nunca calcula fechas, puntajes ni contenido de documentos "de memoria"; todo pasa por código Python verificable.

8.1 Configuración y helpers compartidos

```
import json
import os
import unicodedata
from datetime import date, datetime
from pathlib import Path

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool

load_dotenv()

RUTA_TAREAS = Path(__file__).parent / "tareas.json"

# Carpeta raíz fuera de la cual el agente no puede leer nada (ver Sección 12).
CARPETA_AUTORIZADA = Path(os.getenv("CARPETA_AUTORIZADA",
"C:/MaterialesAcademicos")).resolve()
```

```
EXTENSIONES_ADMITIDAS = {".pdf", ".docx", ".txt", ".md", ".py"}
```

```
PESO_PRIORIDAD = {"alta": 3, "media": 2, "baja": 1}
```

```
def _normalizar(texto: str) -> str:
    """Quita tildes y pasa a minúsculas, para comparar sin importar cómo se
    escriba."""
    sin_tildes = "".join(
        c for c in unicodedata.normalize("NFKD", texto.lower()) if not
        unicodedata.combining(c)
    )
    return sin_tildes.strip()
```

```
def _cargar_tareas() -> list[dict]:
    if not RUTA_TAREAS.exists():
        return []
    return json.loads(RUTA_TAREAS.read_text(encoding="utf-8"))
```

```
def _guardar_tareas(tareas: list[dict]) -> None:
    RUTA_TAREAS.write_text(json.dumps(tareas, ensure_ascii=False, indent=2),
        encoding="utf-8")
```

```
def _buscar_tarea(tarea_id: int) -> dict | None:
    return next((t for t in _cargar_tareas() if t["id"] == tarea_id), None)
```

```
def _validar_ruta(ruta: str) -> Path | None:
    """Devuelve la ruta resuelta solo si está dentro de CARPETA_AUTORIZADA."""
    ruta_resuelta = Path(ruta).resolve()
    if ruta_resuelta == CARPETA_AUTORIZADA or CARPETA_AUTORIZADA in
    ruta_resuelta.parents:
        return ruta_resuelta
    return None
```

```
def _puntaje_urgencia(tarea: dict) -> float:
    """
    puntaje = peso_prioridad * 100 / (días_restantes + 1)

    
$$\text{puntaje} = \frac{\text{peso} \cdot \text{prioridad}}{\text{días restantes} + 1} \times 100$$

    """
```

```

    Heurística simple y determinista para ordenar tareas: no es una
    medida "científica", solo un criterio reproducible para decidir
    qué atender primero (más prioridad + menos tiempo restante = más urgente).
    """
    dias_restantes = max((date.fromisoformat(tarea["fecha_limite"]) -
date.today()).days, 0)
    peso = PESO_PRIORIDAD.get(tarea["prioridad"], 1)
    return peso * 100 / (dias_restantes + 1)

```

8.2 Gestión de tareas

```

@tool
def consultar_tareas() -> str:
    """Devuelve todas las tareas académicas registradas, con su estado
    actual."""
    tareas = _cargar_tareas()
    if not tareas:
        return "No hay tareas registradas."
    return "\n".join(
        f"[{t['id']}] {t['nombre']} - curso: {t['curso']} - vence:
{t['fecha_limite']} "
        f"- prioridad: {t['prioridad']} - estado: {t['estado']}"
        for t in tareas
    )

@tool
def agregar_tarea(
    nombre: str, curso: str, fecha_limite: str, duracion_minutos: int,
    prioridad: str, ruta_contexto: str = "", entregable: str = "",
) -> str:
    """
    Registra una nueva actividad académica pendiente.

    fecha_limite en formato YYYY-MM-DD. prioridad debe ser 'alta',
    'media' o 'baja'. No inventes estos datos: pídeselos al usuario si
    no los mencionó.
    """
    tareas = _cargar_tareas()
    nuevo_id = max((t["id"] for t in tareas), default=0) + 1
    tareas.append({
        "id": nuevo_id, "nombre": nombre, "curso": curso, "fecha_limite":
fecha_limite,
        "duracion_estimada_minutos": duracion_minutos, "prioridad": prioridad,

```

```

        "estado": "pendiente", "ruta_contexto": ruta_contexto, "entregable":
entregable,
    })
    _guardar_tareas(tareas)
    return f"Tarea '{nombre}' registrada con id {nuevo_id}."

```

@tool

```

def calcular_prioridad(tarea_id: int) -> str:
    """
    Calcula el puntaje de urgencia de una tarea (fecha límite + prioridad
    declarada). Nunca decidas la urgencia "a ojo": usa siempre esta
    herramienta antes de recomendar qué tarea atender primero.
    """
    tarea = _buscar_tarea(tarea_id)
    if tarea is None:
        return f"No existe una tarea con id {tarea_id}."
    dias_restantes = max((date.fromisoformat(tarea["fecha_limite"])) -
date.today()).days, 0)
    return (
        f"Tarea {tarea_id} ({tarea['nombre']}): puntaje de urgencia "
        f"{_puntaje_urgencia(tarea):.1f} (prioridad={tarea['prioridad']},
vence en {dias_restantes} día(s))."
    )

```

@tool

```

def generar_plan(minutos_disponibles: int) -> str:
    """
    Distribuye las tareas pendientes (no completadas) dentro del tiempo
    disponible, de mayor a menor puntaje de urgencia, con 10 minutos de
    descanso entre tareas. Úsala después de revisar prioridades con
    calcular_prioridad; no armes el plan a mano.
    """
    pendientes = sorted(
        (t for t in _cargar_tareas() if t["estado"] != "completada"),
        key=_puntaje_urgencia, reverse=True,
    )
    bloques, minutos_usados = [], 0
    for tarea in pendientes:
        duracion = tarea["duracion_estimada_minutos"]
        if minutos_usados + duracion > minutos_disponibles:
            continue
        bloques.append(f"- {tarea['nombre']} ({duracion} min, prioridad
{tarea['prioridad']})")
        minutos_usados += duracion

```

```

        if minutos_usados < minutos_disponibles:
            bloques.append(" - Descanso (10 min)")
            minutos_usados += 10
    if not bloques:
        return "No hay tiempo suficiente para ninguna tarea pendiente con el tiempo disponible."
    return "PLAN:\n" + "\n".join(bloques) + f"\n\nMinutos usados: {minutos_usados} de {minutos_disponibles}."

@tool
def actualizar_estado(tarea_id: int, nuevo_estado: str) -> str:
    """Cambia el estado de una tarea a 'pendiente', 'iniciada' o 'completada'."""
    if nuevo_estado not in {"pendiente", "iniciada", "completada"}:
        return f"Estado '{nuevo_estado}' inválido. Usa: pendiente, iniciada o completada."
    tareas = _cargar_tareas()
    for t in tareas:
        if t["id"] == tarea_id:
            t["estado"] = nuevo_estado
            _guardar_tareas(tareas)
            return f"Tarea {tarea_id} actualizada a estado '{nuevo_estado}'."
    return f"No existe una tarea con id {tarea_id}."

```

8.3 Consulta documental — sin RAG, por palabra clave

```

@tool
def inspeccionar_carpeta(ruta: str) -> str:
    """
    Lista los documentos compatibles (PDF, DOCX, TXT, Markdown, Python) dentro de una carpeta. Solo funciona dentro de CARPETA_AUTORIZADA; cualquier otra ruta se rechaza.
    """
    carpeta = _validar_ruta(ruta)
    if carpeta is None:
        return f"Ruta no autorizada: '{ruta}' está fuera de {CARPETA_AUTORIZADA}."
    if not carpeta.is_dir():
        return f"La ruta '{ruta}' no existe o no es una carpeta."
    archivos = sorted(f.name for f in carpeta.iterdir() if f.suffix.lower() in EXTENSIONES_ADMITIDAS)
    if not archivos:
        return f"No se encontraron documentos compatibles en '{ruta}'."
    return f"Documentos encontrados en '{ruta}': " + ", ".join(archivos)

```

```

def _extraer_texto_archivo(ruta: Path) -> str:
    """Extrae texto plano según la extensión. Sin embeddings: solo lectura."""
    sufijo = ruta.suffix.lower()
    if sufijo in {".txt", ".md", ".py"}:
        return ruta.read_text(encoding="utf-8", errors="ignore")
    if sufijo == ".docx":
        from docx import Document
        return "\n".join(p.text for p in Document(ruta).paragraphs)
    if sufijo == ".pdf":
        from pypdf import PdfReader
        return "\n".join(pagina.extract_text() or "" for pagina in
PdfReader(ruta).pages)
    return ""

```

@tool

```

def buscar_en_documentos(tarea_id: int, consulta: str) -> str:
    """
    Busca líneas que contengan la consulta (coincidencia de palabra
    clave, SIN embeddings ni vector store) dentro de los documentos de
    la carpeta asociada a una tarea. Úsala para saber qué dicen los
    materiales; no asumas ni inventes su contenido.
    """
    tarea = _buscar_tarea(tarea_id)
    if tarea is None:
        return f"No existe una tarea con id {tarea_id}."
    carpeta = _validar_ruta(tarea["ruta_contexto"])
    if carpeta is None or not carpeta.is_dir():
        return f"La carpeta de materiales de la tarea {tarea_id} no es
válida."

    termino = _normalizar(consulta)
    coincidencias = []
    for archivo in carpeta.iterdir():
        if archivo.suffix.lower() not in EXTENSIONES_ADMITIDAS:
            continue
        try:
            texto = _extraer_texto_archivo(archivo)
        except Exception:
            continue
        for linea in texto.splitlines():
            if termino in _normalizar(linea):
                coincidencias.append(f"[{archivo.name}] {linea.strip()}")

```



```
if not coincidencias:
    return f"No se encontró '{consulta}' en los documentos de la tarea {tarea_id}."
return "\n".join(coincidencias[:10])
```

Por qué no hay indexar_documentos ni vector store: con el volumen de archivos de una carpeta de materiales de curso (unos pocos PDF/DOCX por tarea), una búsqueda de texto directa es suficiente y evita la complejidad de embeddings + base vectorial que no aporta valor real a esta escala (ver Sección 15).

9. System prompt (texto real, fuerza el orden de las tools)

```
PROMPT_SISTEMA = """
Eres un agente personal de planificación académica. Ayudas a priorizar
tareas pendientes y a consultar los materiales asociados a cada una,
usando siempre tus herramientas – nunca inventes fechas, prioridades
ni contenido de documentos que no hayas consultado.

Flujo recomendado:
1. Usa consultar_tareas para ver qué hay pendiente.
2. Usa calcular_prioridad sobre las tareas relevantes para decidir cuál
   atender primero (nunca decidas la urgencia "a ojo").
3. Si el usuario da un tiempo disponible, usa generar_plan para
   distribuir las tareas dentro de ese tiempo.
4. Si el usuario quiere empezar una tarea, usa inspeccionar_carpeta
   para ver qué materiales existen, y buscar_en_documentos para
   consultar instrucciones, requisitos o conceptos.
5. Usa actualizar_estado cuando el usuario indique que empezó o
   terminó una tarea, y agregar_tarea cuando mencione una actividad
   nueva que no esté registrada.

Reglas:
– Solo puedes inspeccionar o buscar dentro de carpetas dentro de la
  carpeta autorizada; si el usuario pide otra ruta, recházala.
– Sé breve y concreto: prioridad, plan de tiempo y próximos pasos.
"""
```

10. Selección de proveedor de modelo y creación del agente

Mismo patrón que `agente_presupuesto_materiales.py` de esta sesión (`LLM_PROVIDER`): permite correr sobre Anthropic (de pago) u Ollama (local, gratis) sin tocar el resto del código.

```
LLM_PROVIDER = os.getenv("LLM_PROVIDER", "anthropic").lower()

if LLM_PROVIDER == "ollama":
    MODEL_ID = f"ollama:{os.getenv('OLLAMA_MODEL', 'llama3.2')}"
else:
    MODEL_ID = f"anthropic:{os.getenv('ANTHROPIC_MODEL', 'claude-sonnet-4-6')}"

agent = create_agent(
    model=MODEL_ID,
    system_prompt=PROMPT_SISTEMA,
    tools=[
        consultar_tareas, agregar_tarea, calcular_prioridad, generar_plan,
        actualizar_estado, inspeccionar_carpeta, buscar_en_documentos,
    ],
)
```

Sin memoria de conversación entre sesiones (cada consulta es independiente, como en `agente_presupuesto_materiales.py`): no hace falta un *checkpoint* para este ejercicio.

11. Flujo principal

```
def extraer_texto(contenido) -> str:
    """Convierte la respuesta del modelo (string o lista de bloques) en texto plano."""
    if isinstance(contenido, str):
        return contenido
    return "\n".join(
        bloque.get("text", "") if isinstance(bloque, dict) else
        getattr(bloque, "text", "")
        for bloque in contenido
    )

if __name__ == "__main__":
    print(f"(Usando LLM_PROVIDER={LLM_PROVIDER}, modelo={MODEL_ID})\n")
    while True:
        solicitud = input("Tú: ").strip()
        if solicitud.lower() in {"salir", "exit", "quit"}:
```

```
        break
    resultado = agent.invoke({"messages": [{"role": "user", "content":
solicitud}]})
    print(f"Agente: {extraer_texto(resultado['messages'][-1].content)}\n")
```

12. Consideraciones de seguridad

- CARPETA_AUTORIZADA define la única raíz desde la que se puede leer (_validar_ruta rechaza cualquier ruta fuera de ella, incluidas rutas relativas que intenten escapar con ..).
- Solo se leen las extensiones de EXTENSIONES_ADMITIDAS ; ningún archivo se ejecuta.
- Las tools nunca escriben fuera de tareas.json .

13. Estructura de carpetas

```
C:/MaterialesAcademicos/
├── AgentesIA/TareaReAct/
│   ├── diapositivas.pdf
│   ├── instrucciones.docx
│   └── ejemplo_react.py
└── Investigacion/
    ├── formato_proyecto.docx
    └── metodologia.pdf
```

14. Tipos de archivo admitidos (v1)

PDF (*Portable Document Format*), DOCX (formato de Microsoft Word), TXT, Markdown y archivos Python. PowerPoint, Excel/CSV y la integración con Google Calendar quedan para v2 (Sección 16).

15. Por qué v1 no usa RAG ni vector store

El volumen de materiales por tarea es pequeño (unos pocos documentos por carpeta) y estructurado por carpetas — exactamente el caso donde una búsqueda directa por palabra clave (Sección 8.3) es más simple, más fácil de depurar y suficiente, sin la complejidad adicional de generar embeddings, mantener una base vectorial y gestionar su actualización cuando cambian los archivos. RAG queda como mejora natural para v2 si la cantidad de

materiales creciera lo suficiente como para que la búsqueda semántica aporte valor real sobre la búsqueda literal.

16. Alcance — v1 (esta entrega) vs. v2

v1 (MVP — *Minimum Viable Product*, producto mínimo viable — esta entrega): las 7 tools de la Sección 8, JSON como almacenamiento, búsqueda por palabra clave, sin integraciones externas.

v2 (mejoras futuras, fuera de esta entrega):

- **Integración real con Google Calendar** (vía `google-api-python-client`, `OAuth2`, `credentials.json`) para leer/crear eventos cuando el usuario ya agenda sus tareas ahí — se difiere porque requiere configurar un proyecto en Google Cloud y un flujo de autorización que no es razonable resolver en el plazo de esta tarea.
 - RAG / *vector store* semántico si el volumen de materiales lo justifica (Sección 15).
 - Notificaciones de fechas límite, interfaz web, base de datos en vez de JSON.
-

17. Declaración de transparencia de IA

Este documento y el código que describe fueron elaborados con asistencia de un asistente de IA (Claude Code), a partir de la necesidad personal, los objetivos y las decisiones de alcance definidos por el estudiante (qué va en v1 y qué se difiere a v2, por qué no usar RAG todavía, por qué excluir Google Calendar de esta versión). El asistente ayudó a redactar el detalle técnico, el código de las herramientas y esta especificación.

18. Por qué esto es un agente, de tipo Goal-Based, y dónde está el patrón ReAct

18.1 ¿Por qué es un agente y no una Chain?

Diferencia clave de la Sesión 12: una **Chain** ejecuta un flujo fijo, decidido de antemano por el desarrollador; un **Agent** usa al LLM como motor de razonamiento para decidir, en tiempo real, qué acción tomar y en qué orden. Acá el orden de las tools **no** está *hardcodeado* en Python: el modelo decide en cada turno si necesita `consultar_tareas`, `calcular_prioridad`, `buscar_en_documentos`, etc., según lo que pida el usuario. `create_agent` arma internamente

el ciclo modelo → decide tool → ejecuta tool → vuelve al modelo, hasta que ya no necesita más tools — eso es lo que lo hace un agente, no un script de pasos fijos.

18.2 ¿Por qué Goal-Based, y no las otras tres?

- **No es Simple Reflex:** no reacciona solo al percepto actual con una regla condición-acción; usa datos que persisten entre turnos (`tareas.json`).
- **No es (solo) Model-Based Reflex:** tener un modelo de estado es necesario pero no alcanza para clasificarlo — el agente además evalúa explícitamente una meta ("¿qué tarea conviene atender primero para cumplirla antes de su fecha límite, dentro del tiempo disponible?") y elige acciones (`calcular_prioridad` , `generar_plan`) en función de esa meta, no de una regla fija de estímulo-respuesta.
- **No es Utility-Based:** no hay una función de utilidad ponderando trade-offs entre objetivos en conflicto bajo incertidumbre; el puntaje de urgencia (Sección 8.1) es una heurística determinista simple, no una optimización de utilidad esperada.
- **Es Goal-Based:** cada tool de planificación (`calcular_prioridad` , `generar_plan`) existe precisamente para acercar el estado actual (tareas pendientes) hacia el estado meta (tareas completadas antes de su fecha límite, dentro del tiempo disponible) — la definición operativa de un agente orientado a metas.

18.3 ¿Dónde está el patrón ReAct?

El ciclo **Thought** → **Action** → **Observation** ocurre dentro de `create_agent` , en cada turno de la conversación — no es código adicional que haya que escribir aparte, es el mecanismo interno que ya provee `create_agent` :

```
Thought:      "El usuario quiere saber qué hacer con 4 horas libres;
               primero necesito ver qué tareas hay pendientes."
Action:      consultar_tareas()
Observation:  "3 tareas pendientes: ..."
```



```
Thought:      "Ahora necesito saber cuál es más urgente."
Action:      calcular_prioridad(tarea_id=1)
Observation:  "Puntaje de urgencia: 150.0 (vence en 2 días)."
```



```
Thought:      "Con la prioridad calculada, puedo armar el plan de tiempo."
Action:      generar_plan(minutos_disponibles=240)
Observation:  "PLAN: - Implementar agente ReAct (240 min) ..."
```



```
Thought:      "El usuario quiere ver los materiales de esa tarea."
Action:      inspeccionar_carpeta(ruta=".../TareaReAct")
Observation:  "Documentos encontrados: diapositivas.pdf, instrucciones.docx,
ejemplo_react.py"
```

(el ciclo se repite hasta que el modelo ya no necesita más tools)

Respuesta final: plan priorizado + materiales disponibles.

Cada "Thought" no se imprime como texto separado en la consola (`create_agent` no expone el razonamiento intermedio por defecto), pero es exactamente lo que el LLM resuelve internamente antes de decidir qué tool invocar. Es el mismo ciclo formalizado en la teoría de la Sesión 12:

donde es el historial acumulado de acciones y observaciones hasta el paso , y es la política del modelo — qué acción tomar dado ese historial.

19. Conclusión

El proyecto implementa un agente personal ReAct que prioriza tareas académicas de forma determinista (fecha límite + prioridad) y consulta sus materiales por palabra clave, sin RAG ni integraciones externas en esta primera versión — un alcance acotado y defendible dentro del plazo de la tarea, con Google Calendar y búsqueda semántica planteados explícitamente como evolución de v2.